

step3_advanced_signal_stability

May 27, 2026

1 Step 3: Advanced Signal Extraction and Stability

This notebook adds unsupervised features (PCA + KMeans distances), interaction terms, and evaluates a blended tree ensemble with 5-fold stratified CV and per-fold threshold tuning.

```
[1]: from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    roc_auc_score,
)
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import RobustScaler

RANDOM_STATE = 42
DATA_PATH = Path('DataSet.csv')
TARGET_COL = 'F3924'
ID_COL = 'Unnamed: 0'
LEAKY_FEATURES = ['F3912']

BANK_FEATURES = [
    'F115', 'F321', 'F527', 'F531', 'F670', 'F1692', 'F2082', 'F2122',
    'F2582', 'F2678', 'F2737', 'F2956', 'F3043', 'F3836', 'F3887',
    'F3889', 'F3891', 'F3894',
]
```

```

PLACEHOLDER_VALUES = {-99999999, 99999999, -9999999, 9999999, -999999, 999999,
    ↪ -9999, 9999}
LARGE_ABS_THRESHOLD = 1e7
PLACEHOLDER_MIN_FRAC = 0.002

TOP_MI = 25
TOP_GAP = 25
N_PCA = 3
N_CLUSTERS = 3

```

```

[2]: df = pd.read_csv(DATA_PATH)
print(f'Loaded shape: {df.shape}')

if ID_COL in df.columns:
    df = df.drop(columns=[ID_COL])

y = df[TARGET_COL].astype(int)
X = df.drop(columns=[TARGET_COL], errors='ignore')

if LEAKY_FEATURES:
    X = X.drop(columns=[col for col in LEAKY_FEATURES if col in X.columns],
    ↪ errors='ignore')

X = X.replace(list(PLACEHOLDER_VALUES), np.nan)
X = X.replace([np.inf, -np.inf], np.nan)
X = X.apply(pd.to_numeric, errors='coerce')

def detect_placeholder_values(frame: pd.DataFrame, abs_threshold: float,
    ↪ min_frac: float) -> dict[str, float]:
    placeholder_map: dict[str, float] = {}
    for col in frame.columns:
        series = frame[col].dropna()
        if series.empty:
            continue
        extreme = series[series.abs() >= abs_threshold]
        if extreme.empty:
            continue
        counts = extreme.value_counts()
        candidate = counts.index[0]
        if counts.iloc[0] / len(series) >= min_frac:
            placeholder_map[col] = candidate
    return placeholder_map

placeholder_map = detect_placeholder_values(X, LARGE_ABS_THRESHOLD,
    ↪ PLACEHOLDER_MIN_FRAC)
for col, value in placeholder_map.items():
    X[col] = X[col].replace(value, np.nan)

```

```

print('Features after cleaning:', X.shape)
print('Placeholder columns detected:', len(placeholder_map))
print('Target base rate:', y.mean())

```

Loaded shape: (9082, 3925)
 Features after cleaning: (9082, 3922)
 Placeholder columns detected: 76
 Target base rate: 0.008918740365558247

```

[3]: def build_row_stats(frame: pd.DataFrame) -> pd.DataFrame:
    values = frame.to_numpy(dtype=float)
    mask = ~np.isnan(values)
    non_missing = mask.sum(axis=1)
    total = values.shape[1]
    missing_rate = 1.0 - (non_missing / total)

    zero_rate = np.where(non_missing > 0, (values == 0).sum(axis=1) /
↳non_missing, 0)
    positive_rate = np.where(non_missing > 0, (values > 0).sum(axis=1) /
↳non_missing, 0)
    negative_rate = np.where(non_missing > 0, (values < 0).sum(axis=1) /
↳non_missing, 0)

    with np.errstate(all='ignore'):
        mean = np.nanmean(values, axis=1)
        std = np.nanstd(values, axis=1)
        min_val = np.nanmin(values, axis=1)
        max_val = np.nanmax(values, axis=1)
        median = np.nanmedian(values, axis=1)
        q25 = np.nanpercentile(values, 25, axis=1)
        q75 = np.nanpercentile(values, 75, axis=1)
        abs_mean = np.nanmean(np.abs(values), axis=1)

    iqr = q75 - q25

    return pd.DataFrame({
        'row_non_missing_count': non_missing,
        'row_missing_rate': missing_rate,
        'row_zero_rate': zero_rate,
        'row_positive_rate': positive_rate,
        'row_negative_rate': negative_rate,
        'row_mean': mean,
        'row_std': std,
        'row_min': min_val,
        'row_max': max_val,
        'row_median': median,

```

```

        'row_q25': q25,
        'row_q75': q75,
        'row_iqr': iqr,
        'row_abs_mean': abs_mean,
    }, index=frame.index)

row_stats = build_row_stats(X)
row_stats.head()

```

```

[3]:   row_non_missing_count  row_missing_rate  row_zero_rate  row_positive_rate  \
0                2728          0.304437      0.488636      0.294355
1                2776          0.292198      0.482709      0.293948
2                2780          0.291178      0.451439      0.330576
3                2900          0.260581      0.423793      0.378966
4                2788          0.289138      0.463773      0.330703

      row_negative_rate      row_mean      row_std  row_min      row_max  \
0          0.217009  17724.644139  243015.108810     -1.00  5933313.82
1          0.223343  15757.835375  230671.462377     -1.00  9133588.45
2          0.217986   6140.730902   46102.216009     -1.02  1093319.56
3          0.197241  29135.495296  246015.394471     -1.00  4458592.60
4          0.205524   7528.900142   62142.156960     -1.19  1154858.43

      row_median  row_q25  row_q75  row_iqr  row_abs_mean
0           0.0      0.0  0.530000  0.530000  17725.036867
1           0.0      0.0  0.400000  0.400000  15758.246823
2           0.0      0.0  0.978333  0.978333   6141.129866
3           0.0      0.0  1.000000  1.000000  29135.851875
4           0.0      0.0  0.768529  0.768529   7529.261570

```

```

[4]: bank_features = [col for col in BANK_FEATURES if col in X.columns and X[col].
    ↪notna().any()]

mi_candidates = X.loc[:, X.nunique(dropna=True) > 1]
top_mi_cols = []
if mi_candidates.shape[1] > 0:
    mi_imputer = SimpleImputer(strategy='median')
    mi_values = mi_imputer.fit_transform(mi_candidates)
    mi_scores = mutual_info_classif(mi_values, y, random_state=RANDOM_STATE)
    mi_series = pd.Series(mi_scores, index=mi_candidates.columns).
    ↪sort_values(ascending=False)
    top_mi_cols = mi_series.head(TOP_MI).index.tolist()

missing_pos = X.loc[y == 1].isna().mean()
missing_neg = X.loc[y == 0].isna().mean()
missing_gap = (missing_pos - missing_neg).abs().sort_values(ascending=False)
top_gap_cols = missing_gap.head(TOP_GAP).index.tolist()

```

```

selected_cols = []
for col in bank_features + top_mi_cols + top_gap_cols:
    if col not in selected_cols:
        selected_cols.append(col)

missing_flags = X[top_gap_cols].isna().astype(int).add_prefix('miss_')
compact_df = pd.concat([X[selected_cols], row_stats, missing_flags], axis=1)
compact_df = compact_df.loc[:, compact_df.isna().mean() < 1.0]

print('Bank features kept:', len(bank_features))
print('Top MI columns:', len(top_mi_cols))
print('Top missingness-gap columns:', len(top_gap_cols))
print('Compact feature shape:', compact_df.shape)

```

Bank features kept: 16
Top MI columns: 25
Top missingness-gap columns: 25
Compact feature shape: (9082, 104)

```

[5]: def add_unsupervised_features(compact_frame: pd.DataFrame, n_pca: int = 3,
    ↪n_clusters: int = 3) -> pd.DataFrame:
    df_out = compact_frame.copy()
    temp_imputed = SimpleImputer(strategy='median').fit_transform(df_out)
    scaled_data = RobustScaler().fit_transform(temp_imputed)

    pca = PCA(n_components=n_pca, random_state=RANDOM_STATE)
    pca_coords = pca.fit_transform(scaled_data)
    for i in range(n_pca):
        df_out[f'feature_pc_{i + 1}'] = pca_coords[:, i]

    kmeans = KMeans(n_clusters=n_clusters, random_state=RANDOM_STATE, n_init=10)
    distances = kmeans.fit_transform(scaled_data)
    for i in range(n_clusters):
        df_out[f'feature_kmeans_dist_c_{i + 1}'] = distances[:, i]

    return df_out

compact_df_enhanced = add_unsupervised_features(compact_df, n_pca=N_PCA,
    ↪n_clusters=N_CLUSTERS)
print('Enhanced feature matrix shape:', compact_df_enhanced.shape)

```

C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by

```

setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
warnings.warn(
    File "C:\Users\amart\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.
10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-
packages\joblib\externals\loky\backend\context.py", line 257, in
_count_physical_cores
    cpu_info = subprocess.run(
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 503, in run
        with Popen(*popenargs, **kwargs) as process:
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 971, in __init__
        self._execute_child(args, executable, preexec_fn, close_fds,
    File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.10_3.10.3
056.0_x64__qbz5n2kfra8p0\lib\subprocess.py", line 1456, in _execute_child
        hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
Enhanced feature matrix shape: (9082, 110)

```

```

[6]: def add_interaction_features(df: pd.DataFrame, top_mi_list: list[str],
    row_stats_list: list[str]) -> pd.DataFrame:
    df_out = df.copy()
    for mi_col in top_mi_list:
        if mi_col in df_out.columns:
            for stat_col in row_stats_list:
                if stat_col in df_out.columns:
                    feat_name = f'interact_{mi_col}_x_{stat_col}'
                    df_out[feat_name] = df_out[mi_col] * df_out[stat_col]
    return df_out

target_mi = top_mi_cols[:3]
target_stats = ['row_missing_rate', 'row_std', 'row_zero_rate']

compact_df_enhanced = add_interaction_features(compact_df_enhanced, target_mi,
    target_stats)
print('Final enhanced feature matrix shape:', compact_df_enhanced.shape)

```

Final enhanced feature matrix shape: (9082, 119)

```

[7]: def report_metrics(name: str, y_true: pd.Series, y_prob: np.ndarray, threshold:
    float = 0.5) -> dict:
    pr_auc = average_precision_score(y_true, y_prob)
    roc_auc = roc_auc_score(y_true, y_prob)
    y_pred = (y_prob >= threshold).astype(int)
    macro_f1 = f1_score(y_true, y_pred, average='macro')
    minority_f1 = f1_score(y_true, y_pred, pos_label=1)
    balanced_acc = balanced_accuracy_score(y_true, y_pred)
    cm = confusion_matrix(y_true, y_pred)

```

```

print(f'=== {name} Performance ===')
print(f'PR-AUC:          {pr_auc:.4f}')
print(f'ROC-AUC:          {roc_auc:.4f}')
print(f'Macro F1-Score: {macro_f1:.4f}')
print(f'Minority F1:      {minority_f1:.4f}')
print(f'Balanced Acc:      {balanced_acc:.4f}')
print('')
print('Confusion Matrix:')
print(cm)
print('')
print('Detailed Report:')
print(classification_report(y_true, y_pred, digits=4))
print('=' * 30)
print('')

return {
    'pr_auc': pr_auc,
    'roc_auc': roc_auc,
    'macro_f1': macro_f1,
    'minority_f1': minority_f1,
    'balanced_acc': balanced_acc,
}

```

```

[8]: try:
        import xgboost as xgb
    except ImportError:
        xgb = None
        print('xgboost not installed. Install with: pip install xgboost')

    try:
        import lightgbm as lgb
    except ImportError:
        lgb = None
        print('lightgbm not installed. Install with: pip install lightgbm')

    try:
        from imblearn.over_sampling import SMOTE
        has_imblearn = True
    except ImportError:
        has_imblearn = False
        print('imbalanced-learn not installed. Install with: pip install_
↳imbalanced-learn')

    def build_xgb(scale_pos_weight: float):
        return xgb.XGBClassifier(
            n_estimators=500,

```

```

        max_depth=5,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        eval_metric='aucpr',
        scale_pos_weight=scale_pos_weight,
        tree_method='hist',
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )

def build_lgb(scale_pos_weight: float):
    return lgb.LGBMClassifier(
        n_estimators=500,
        learning_rate=0.05,
        num_leaves=31,
        subsample=0.8,
        colsample_bytree=0.8,
        objective='binary',
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_STATE,
        n_jobs=-1,
        verbosity=-1,
    )

```

```

[9]: if xgb is None or lgb is None or not has_imblearn:
    print('Missing xgboost/lightgbm/imbalanced-learn. Install the packages to_
    ↪run CV.')
else:
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    fold_metrics = []

    for fold, (train_idx, val_idx) in enumerate(skf.split(compact_df_enhanced,
    ↪y), start=1):
        X_tr = compact_df_enhanced.iloc[train_idx]
        y_tr = y.iloc[train_idx]
        X_va = compact_df_enhanced.iloc[val_idx]
        y_va = y.iloc[val_idx]

        imputer = SimpleImputer(strategy='median')
        X_tr_imp = imputer.fit_transform(X_tr)
        X_va_imp = imputer.transform(X_va)

        smote = SMOTE(sampling_strategy=0.1, random_state=RANDOM_STATE)
        X_tr_res, y_tr_res = smote.fit_resample(X_tr_imp, y_tr)

        spw = (y_tr_res == 0).sum() / max(y_tr_res.sum(), 1)

```



```

xgb_fold = build_xgb(spw).fit(X_tr_res, y_tr_res)
lgb_fold = build_lgb(spw).fit(X_tr_res, y_tr_res)

p_xgb = xgb_fold.predict_proba(X_va_imp)[: , 1]
p_lgb = lgb_fold.predict_proba(X_va_imp)[: , 1]
blend_probs = (0.6 * p_xgb) + (0.4 * p_lgb)

prec, rec, thrs = precision_recall_curve(y_va, blend_probs)
f1s = (2 * prec[:-1] * rec[:-1]) / (prec[:-1] + rec[:-1] + 1e-12)
opt_idx = int(np.argmax(f1s))
opt_thr = thrs[opt_idx]

y_pred = (blend_probs >= opt_thr).astype(int)
fold_macro_f1 = f1_score(y_va, y_pred, average='macro')
fold_minority_f1 = f1_score(y_va, y_pred, pos_label=1)
fold_pr_auc = average_precision_score(y_va, blend_probs)

fold_metrics.append([fold_pr_auc, fold_macro_f1, fold_minority_f1])
print(
    f'Fold {fold} | PR-AUC: {fold_pr_auc:.4f} | Macro F1:␣
↪{fold_macro_f1:.4f} '
    f'| Minority F1: {fold_minority_f1:.4f} | Optimal Thr: {opt_thr:.
↪4f}'
)

cv_summary = pd.DataFrame(fold_metrics, columns=['PR-AUC', 'Macro F1',␣
↪'Minority F1'])
print('')
print('=== 5-Fold Cross-Validation Summary ===')
print(cv_summary.mean().to_frame('Mean').join(cv_summary.std().
↪to_frame('Std')))
```

Fold 1 | PR-AUC: 0.8541 | Macro F1: 0.8992 | Minority F1: 0.8000 | Optimal Thr: 0.3161

Fold 2 | PR-AUC: 0.9306 | Macro F1: 0.9369 | Minority F1: 0.8750 | Optimal Thr: 0.5068

Fold 3 | PR-AUC: 0.8317 | Macro F1: 0.9280 | Minority F1: 0.8571 | Optimal Thr: 0.9912

Fold 4 | PR-AUC: 0.9403 | Macro F1: 0.9328 | Minority F1: 0.8667 | Optimal Thr: 0.3850

Fold 5 | PR-AUC: 0.7702 | Macro F1: 0.8574 | Minority F1: 0.7179 | Optimal Thr: 0.0400

=== 5-Fold Cross-Validation Summary ===

	Mean	Std
PR-AUC	0.865385	0.071022
Macro F1	0.910870	0.033336

Minority F1 0.823352 0.065842